Goals of project

In scope

For Pixar, the primary focus is Vulkan rendering on Linux with full-featured parity with OpenGL.

For Autodesk, the primary focus is to get Vulkan rendering on Windows (Vulkan 1.3) and Android 7 (and higher) (Vulkan 1.0 - 1.3) with full-featured parity with OpenGL and Metal.

For Pixar and Autodesk, an important milestone of the HgiVulkan work will be the ability to run all relevant tests (i.e. those within the hdSt, hdx, usdImagingGL, etc. directories) with a “passing” result. A feature-complete usdview experience is additionally important. We also expect performance marginally equal or better than the current performance of Storm using the HgiGL backend.

Pixar wants to be able to run native Vulkan usdview, but also support interop into a GL framebuffer. Autodesk also wants to be able to run native Vulkan usdview, however prioritizes the use of a Vulkan based swap-chain management, presentation and composition system over GL based interop.

Autodesk wants to regularly port updates/progress made to the Vulkan backend to Android (OS: 7 and higher, Vulkan 1.0 and 1.3). The best path for merging Android fixes can be chosen by Pixar and Autodesk.

Adobe wants to support:

- Linux (desktop)
- Windows (desktop)
- Android (high end)

A summary of these goals:

Functionality	Pixar	Autodesk	Adobe
Linux	Vulkan 1.3 rendering on Linux with full-featured parity with OpenGL	Same	Same
Windows	Not an immediate priority	Vulkan 1.3 rendering on Windows with full-feature parity with OpenGL and Metal	Same as Autodesk
Android	Not an immediate priority	Vulkan 1.0-1.3 rendering on Android 7 (and higher) with full-feature parity with OpenGL and Metal Regularly port updates/progress made to the Vulkan backend to Android (OS: 7 and higher, Vulkan 1.0 and 1.3).	High end android devices and headsets are important, covering older devices are less critical (eg Meta Quest and higher end tablets)
Mac	Not an immediate priority	We'll use Metal	We'll use Metal
Tests	All relevant tests (i.e. those within the hdSt, hdx, usdImagingGL, etc. directories) run with a "passing" result	Same	Same
Performance	HgiVulkan should be marginally equal -- ideally better than -- the current performance of Storm using the HgiGL backend	Same	Same
Native Vulkan usdview	Be able to run native Vulkan usdview, but also support interop into a GL framebuffer	Be able to run native Vulkan usdview, but prioritize the use of a Vulkan based swap-chain management, presentation and composition system over GL based interop	No major concerns with Usdview approach, but we want it to support our renderer (which is a Hydra delegate) with high performance without forking USD

Out of scope

Getting to parity with the current functionality of HdStorm with GL is the main goal of this project. Additional features beyond the current feature set (e.g. ray-tracing with Vulkan) are potentially of interest in the future but are out of scope for this project.

Technical Details

Both Pixar and Autodesk currently have internal versions of the OpenUSD repository with a partially functional implementation of HgiVulkan. An initial goal will be to resolve any differences between those trees, likely through the use of a feature branch on the OpenUSD repository. HgiVulkan developers will be able to share code and iterate faster via use of this feature branch. As code changes are tested and approved, they can be checked into Pixar's internal Perforce tree by Pixar developers, and thus be available within the OpenUSD dev branch soon after.

Even considering those partially functional implementations, there are a number of bugs and missing features to fix before reaching full functional parity with OpenGL.

Testing

Tests (such as those within the hdSt, hdx, and usdImagingGL directories) are used by Pixar developers to confirm working feature functionality; a few of these tests are available in the open source OpenUSD repository, though many are not. Given the work in Pixar's internal HgiVulkan tree, most of the tests within the aforementioned directories are currently "passing", where passing is defined as the test result having differences that are indistinguishable to the human eye or are noticeable but are considered negligible or an improvement on the existing baselines. Though not all, the vast majority of the (seemingly correct) images produced by Storm with HgiVulkan show small differences from their OpenGL counterparts. As more of the HgiVulkan work is checked into Pixar's Perforce depot and the HgiVulkan backend becomes increasingly viable internally, Pixar will need to determine how to best handle those differing HgiVulkan test results.

Pixar would also like to open-source more Hydra and usdImagingGL tests (like those mentioned above).

More specific HgiVulkan test coverage may need to be added (some new test coverage is already included in the Autodesk HgiVulkan proposal). Tests that are GL/HgiGL specifc (such as testHdStIndirectDrawBatchCodeGen, as the indirect draw batch does not work with HgiVulkan) should be modified or given an HgiVulkan/Hgi backend-agnostic counterpart.

Manual testing using usdview, particularly on large and complex assets, will likely become increasingly important as basic functionality and the functionality covered by tests nears completion.

DX12 and WebGPU considerations for Vulkan work

Architectural decisions to support Vulkan will be made in context of ongoing discussions about how to support DirectX 12 and WebGPU in Hgi, although that work isn't part of this proposal.

Adobe Notes

Our use case is an application comprised of different modules:

1. a Hydra render delegate using low level Vulkan/Metal for raytracing and other cutting edge capabilities to render on the GPU
2. a middleware framework based around USD/Hydra providing foundational 3D capabilities
3. a front-end UI based on Qt Quick or other GPU accelerated technology

We want to be able to share framebuffers directly on the GPU between the components that can run different Vulkan instances. GPU to GPU copy can be made to decouple the synchronization between the modules, but at no point do we want the framebuffers to transit through the CPU memory. Note that this situation is not necessarily how we want to do it, it's just the reality of all the involved components are treating Vulkan as if they are the only Vulkan consumer in the process. This makes it hard to just pass in an external Vulkan instance and expect it to work (they have different extension requirements, queue needs, etc.). The reality is that different frameworks and components are not necessarily easy to make share Vulkan instance and interop support is a way of lowering this barrier to adoption.

Our current approach:

We modify USD to allow wrapping existing native textures (Vulkan, Metal) into an HgiTexture object, that we then copy on the GPU to a custom shared texture object that is exportable to other Vulkan instances using the *external memory* native handle extensions. On Metal, texture ID can be shared directly on the same device. This requires ABI breaking changes to USD HgiTexture we'd like to avoid.

What we would like to see:

1. HgiTexture should offer at the minimum the ability to wrap an existing GPU resource from the same instance (VkImage, Metal TextureId, OpenGL ID) inside an HgiTexture.
2. Ideally for HgiVulkan to fully support external memory interop by enabling the creation of shared memory allocation pools and exporting native handles (win32 opaque, or linux FD handles). Through the HgiTexture API.

Differing approaches

- Multiple Vulkan instances or some shared understanding of single Vulkan instance that is shared between applications
    - Single Vulkan instance: https://github.com/PixarAnimationStudios/OpenUSD/pull/2553